

DOCUMENTAÇÃO TÉCNICA

Modelo para o Projeto Integrador DelBicos

1 Identificação do Projeto

1.1. Nome do Projeto: DelBicos – Delivery de Bicos

1.2. Grupo/Equipe:

- Douglas Wenzel, 3011392413022
- Eduardo Kamo, 3011392413005
- Fernando Chibli, 3011392323017
- Fernando Ribeiro, 3011392413020
- Gustavo Ferreira, 3011392413016
- Iago Rossan, 3011392413038
- Isabel Maito, 3011392413045
- Lucas Consani, 3011392413046

1.3. Data da Elaboração: 22/08/2026

1.4. Versão do Documento: 2.1 [16/09/2026] – seções 6, 8, 11 e 13 revisadas após confirmar o estado da branch stag-facul

2 Visão Geral do Produto

2.1. Problema a ser Resolvido: Profissionais autônomos que prestam pequenos serviços domésticos e prediais (limpeza, pequenos reparos, montagem de móveis, elétrica etc. — os “bicos”) têm dificuldade para divulgar seu trabalho, organizar a agenda e comprovar renda de forma profissional. Do lado do cliente, falta um canal único e confiável para encontrar, comparar, agendar e pagar esses profissionais, com garantias mínimas de segurança (avaliações, pagamento intermediado, comunicação registrada).

2.2. Solução Proposta: O DelBicos é um marketplace multiplataforma (app mobile Android/iOS e versão web, mesmo código-base em React Native + Expo) que conecta clientes e profissionais autônomos. Oferece busca de profissionais por categoria/subcategoria e geolocalização (raio de atuação); perfil do profissional com

portfólio, comodidades e avaliações; agendamento respeitando a disponibilidade real cadastrada; pagamento por cartão via Stripe, com o agendamento só sendo criado após a confirmação do pagamento; chat em tempo real (Socket.IO) por agendamento; notificações push (Expo); e, em desenvolvimento, um chatbot com Processamento de Linguagem Natural e comando de voz para agendar, consultar e cancelar serviços por texto ou fala.

2.3. Público-Alvo: Duas personas principais — (1) Cliente residencial/comercial que precisa contratar um “bico” pontual e quer rapidez, preço claro e segurança para deixar um desconhecido em sua casa; (2) Profissional autônomo ou pequena prestadora de serviços que quer divulgar seu trabalho, controlar a própria agenda e receber pagamentos sem burocracia. Uma terceira persona, o Administrador da plataforma, modera categorias e profissionais e acompanha métricas gerais (dashboard admin).

3 Requisitos Funcionais e Não Funcionais

3.1. Requisitos Funcionais (RFs): RF01 Alternância de Perfil – trocar entre a visão Cliente e Parceiro por um switch no menu. RF02 Onboarding do Parceiro – cadastro de dados profissionais (especialidades, biografia, documentos). RF03 Gestão de Bicos – CRUD de serviços oferecidos pelo profissional. RF04 Filtro de Geolocalização – o parceiro define um raio de atuação (km) e só recebe pedidos dentro dele. RF05 Gestão de Status do Agendamento – fluxo Aceitar (confirmed) → A caminho (on the way) → Iniciar (in progress) → Concluir (completed). RF06 Agenda Dinâmica – visualização dos agendamentos confirmados em lista/calendário. RF07 Chat Contextual – canal de texto entre cliente e parceiro após a confirmação do agendamento. RF08 Check-out de Segurança – confirmação para finalizar o serviço. RF09 Portfólio (mídia) – upload de fotos dos trabalhos realizados. RF10 Relatórios de Ganhos – exportação do histórico de serviços e valores. Também já implementados: autenticação/cadastro com verificação de e-mail, favoritos, avaliações (rating + review) e dashboard de KPIs do profissional; e, em desenvolvimento, agendamento por PLN/voz (chatbot).

3.2. Requisitos Não Funcionais (RNFs): RNF01 Responsividade – interface adaptável de smartphone a tablet (5" a 11") e à web. RNF02 Persistência Local – armazenamento local (AsyncStorage) para agenda e rascunhos offline. RNF03 Sincronização – dados locais sobem para o backend assim que há conexão. RNF04 Segurança JWT – controle de acesso por papel (Cliente, Parceiro, Admin) em todos os endpoints autenticados. RNF05 Performance de Mídia – compressão de imagens antes do upload. RNF06 Validação Robusta – validação no front (React Hook Form) e no backend (middlewares Express). RNF07 Sincronismo em Nuvem – alterações no mobile devem refletir no backend em menos de 2s (REST + Socket.IO). RNF08 Acessibilidade – suporte a VLibras e contraste no modo escuro/alto contraste. Cada RF está associado a pelo menos um RNF (ex.: RF05 depende de RNF07 para refletir status em tempo real nos dois lados).

4 Arquitetura do Sistema

4.1. Visão Geral da Arquitetura: Arquitetura cliente-servidor com um backend único (monólito modular) em Node.js/Express expondo uma API REST, complementado por um serviço Python (nlp-service) dedicado a PLN/embeddings, consumido internamente pela API Node. O frontend é um único app React Native (Expo) que compila para mobile (Android/iOS) e para web (React Native Web), consumindo a mesma API. Comunicação em tempo real (chat e status de agendamento) via Socket.IO no mesmo servidor HTTP; atualização do catálogo de serviços via Server-Sent Events (SSE). Bancos políglotas: PostgreSQL (Sequelize) para as entidades de negócio e MongoDB (Mongoose) para mensagens de chat e notificações. Fluxo: App Mobile/Web ⇌ REST/JWT + Socket.IO ⇌ API Express/TypeScript ⇌ {PostgreSQL, MongoDB, nlp-service, Stripe, SendGrid, S3/ImgBB, Expo Push}.

4.2. Componentes Principais: Frontend – React Native 0.81 + Expo 54, TypeScript, Zustand 5 (estado), React Navigation 7, Axios, Stripe React Native/JS, React Native Maps. Backend – Node.js ≥18, Express 4.21, TypeScript 5, Sequelize 6.37 (PostgreSQL/MySQL), Mongoose 8.14 (MongoDB), Socket.IO 4.8, JWT (jsonwebtoken 9). Serviço de PLN (nlp-service) – Python, scikit-learn (TF-IDF + LinearSVC) para classificação de intenções e sentence-transformers (paraphrase-multilingual-MiniLM-L12-v2) para busca semântica, NLTK para pré-processamento/stemming em português. Integrações – Stripe (pagamentos), SendGrid (e-mail transacional), AWS S3/ImgBB (mídia), Expo Push (notificações), AWS Lambda (fallback de e-mail), Winston + Logtail/CloudWatch (logs).

4.3. Tecnologias Utilizadas: TypeScript 5.3+ (front e back), Node.js ≥18, Express 4.21.2, Sequelize 6.37.8, Mongoose 8.14.1, PostgreSQL/MySQL, MongoDB, Socket.IO 4.8.3, React Native 0.81.5, Expo SDK 54, React 19.1, Zustand 5.0.5, React Navigation 7, Jest 29/30 + Supertest (testes), Docker/Docker Compose, GitHub Actions (CI/CD), Python 3 + scikit-learn + sentence-transformers + NLTK (nlp-service), Stripe, SendGrid, Winston.

5 Modelo de Dados

5.1. Diagrama de Entidade-Relacionamento (DER): O modelo relacional (PostgreSQL, gerido por migrations Sequelize) tem User como entidade central, especializada em Client, Professional e Admin (1-para-1 por user_id). Um Professional oferece vários Service (agrupados por Category → Subcategory), define Availability e AvailabilityLock (bloqueios pontuais de agenda) e recebe Appointment de um Client em um Address. Cada Appointment pode gerar um ChatRoom (chat contextual) e grava rating/review quando avaliado. Favorite liga Client a Professional; Notification e ChatMessage ficam em MongoDB. O diagrama completo (entidades e FKs) deve ser exportado do próprio banco (ex.: sequelize-erd) ou desenhado no dbdiagram.io a partir da lista de entidades da seção 5.2 e anexado como imagem.

5.2. Descrição das Entidades Principais: User: id (PK), name, email (único), phone (único), password (hash bcrypt), active, avatar_uri. Client: id (PK), user_id (FK), main_address_id (FK), cpf (único). Professional: id (PK), user_id (FK),

main_address_id (FK), cpf (único), cnpj (opcional), description, raio de atuação. Address: id (PK), user_id (FK), street, number, neighborhood, city, state, postal_code, lat, lng. Category: id (PK), title, description, active. Subcategory: id (PK), title, category_id (FK), active. Service: id (PK), title, description, price (decimal), duration (min), subcategory_id (FK), professional_id (FK), active. ProfessionalAvailability/ServiceAvailability: id (PK), professional_id/service_id (FK), day_of_week (0-6), start_time, end_time. ProfessionalAvailabilityLock: id (PK), professional_id (FK), lock_date, start_time, end_time, reason. Appointment: id (PK), professional_id, client_id, service_id, address_id (FKs), start_time, end_time, status (pending/confirmed/completed/canceled), final_price, rating (1-5), review, payment_intent_id. Favorite: id (PK), user_id, professional_id (FKs). ChatRoom (Postgres): id (PK), appointment_id, professional_id, client_id, service_id (FKs), status. ChatMessage/Notification: coleções MongoDB (_id ObjectId), referenciando os IDs relacionais.

6 APIs (Application Programming Interfaces)

6.1. Design da API: API RESTful (JSON sobre HTTPS), documentada via Swagger/OpenAPI em /docs, complementada por WebSocket (Socket.IO) e Server-Sent Events (catálogo de serviços, GET /api/services/events). Principais grupos de rotas (base /api, exceto /auth): auth (POST /auth/register, /verify, /resend), user (POST /api/user/login, GET/PUT /api/user/me, POST /api/user/change-password), address (CRUD), categories/subcategories (leitura pública), professionals (busca com filtros de termo/geolocalização, cadastro/edição), services (CRUD completo, incluindo GET /api/services/my), professionalAvailability e availability-locks (CRUD de agenda), appointments (criar, listar, PATCH /:id/confirm, PUT /:id/status, POST /:id/review, GET /:id/receipt), payments (create-payment-intent e confirm via Stripe), favorites (GET/POST/DELETE), notifications (listar/marcar lidas, salvar token Expo), dashboard (KPIs, ganhos, serviços por categoria), chat (GET /rooms, GET /rooms/:roomId/messages), admin (login e estatísticas). Já implementados na branch stag-facul (base usada para as entregas da faculdade, ainda não mesclada à main de produção): chatbot com PLN (POST /api/chat/bot/message, GET /api/chat/bot/session/active e /:id) e comando de voz/busca semântica (POST /api/voice/transcriptions, POST /api/voice/commands, GET /api/services/search/semantic).

6.2. Autenticação e Autorização: Autenticação stateless via JWT, emitido no login (/api/user/login e /api/admin/login) e enviado em Authorization: Bearer <token> em toda requisição privada. O middleware auth.middleware.ts valida o token e injeta req.user/req.client/req.address; admin.middleware.ts restringe rotas administrativas. Rate limiting (express-rate-limit) protege rotas sensíveis (login, chatbot, comando de voz). Middlewares de segurança adicionais (helmet, hpp, mongo-sanitize, sanitização XSS, guarda de SQL injection) já existem no código (security.middleware.ts), mas estão comentados/desativados em server.ts — recomenda-se reativá-los antes do deploy final (ver seção 11, Riscos). Senhas usam hash bcrypt, nunca texto puro.

7 Computação em Nuvem (Azure)

7.1. Serviços Azure Utilizados: Hoje o backend roda em uma instância AWS EC2 (Docker) com deploy via GitHub Actions (build → Docker Hub → SSH deploy); mídia usa AWS S3/ImgBB e e-mail usa SendGrid + fallback AWS Lambda. Para esta disciplina, propõe-se migrar/espelhar a hospedagem para a Azure com: Azure App Service ou Container Apps (API Node containerizada); Azure Database for PostgreSQL – Flexible Server; Azure Cosmos DB (API MongoDB) para chat/notificações; Azure Blob Storage para mídia; Azure Container Registry (imagens da API e do nlp-service); Azure Static Web Apps para o build web do Expo; Azure Cognitive Services Speech-to-Text como alternativa de transcrição de voz; Application Insights + Azure Monitor para observabilidade; Azure Key Vault para segredos (JWT, Stripe, SendGrid).

7.2. Arquitetura de Deploy: CI/CD via GitHub Actions: um workflow builda e publica a imagem Docker da API; o workflow de deploy (hoje via SSH para EC2) seria adaptado para publicar no Azure Container Registry e disparar o deploy no App Service/Container Apps (Azure CLI ou action azure/webapps-deploy). O nlp-service (Python) seria publicado como um segundo Container App, acessado internamente pela API Node por rede privada. As migrations do Sequelize rodam como job único antes do rollout de cada nova versão. Segredos migram para o Azure Key Vault, referenciados pelo App Service. O frontend web (build Expo Web) é publicado como Azure Static Web App, com preview automático por Pull Request.

7.3. Monitoramento e Logs: Já existe logging estruturado com Winston, exportado para Logtail e opcionalmente AWS CloudWatch. Na versão Azure, propõe-se centralizar em Application Insights (requisições HTTP, exceptions, dependências como Stripe/SendGrid/nlp-service) e Azure Monitor (métricas de CPU/memória/latência e alertas, por exemplo taxa de erro 5xx acima de 5% em 5 minutos).

7.4. Estimativa de Custos: Estimativa mensal aproximada (região Brazil South, uso baixo/MVP acadêmico, a validar na calculadora oficial da Azure): App Service B1 (API) ≈ US\$ 13; Container App do nlp-service (scale-to-zero) ≈ US\$ 5–15; PostgreSQL Flexible Server Burstable B1ms ≈ US\$ 15; Blob Storage (poucos GB) ≈ US\$ 1–3; Static Web App (frontend web) – camada gratuita; Application Insights – camada gratuita/baixo custo; Key Vault – custo desprezível. Total estimado: ≈ US\$ 35–50/mês para um ambiente de demonstração.

8 Qualidade e Testes de Software

8.1. Estratégia de Testes: Testes unitários para regras de negócio isoladas (services, utils, parsers de data/PLN) e testes de integração para os endpoints REST mais críticos (autenticação, agendamento, pagamento, chat) com Supertest contra o app Express e banco de teste isolado. Testes manuais/exploratórios cobrem os fluxos ponta a ponta no app antes de cada entrega. Planeja-se ainda testes de caixa preta (Partição de Equivalência/Análise de Valor Limite) e um diagrama de estados com

cobertura de estados/transições para o fluxo de agendamento (RF05), conforme exigido pela disciplina satélite de Qualidade e Testes de Software.

8.2. Ferramentas de Teste: Backend – Jest 29/30 + ts-jest + Supertest (unitário e integração), com projetos separados (test:unit, test:integration) em jest.config.js; pytest no nlp-service. Frontend – jest-expo 54 para componentes/hooks. CI – GitHub Actions executa lint/build/testes a cada push (docker-ci.yml).

8.3. Cobertura de Testes: Na branch main (produção) a cobertura efetiva ainda é baixa — apenas src/services/__tests__/payment.service.test.ts no backend. Já na stag-facul (base usada para a entrega da faculdade), o chatbot e o comando de voz trouxeram dezenas de arquivos de teste (NLU, estados do bot, roteador de mensagens, gravação de voz, idempotência, utilitário de datas), elevando bastante a cobertura nessa branch — 146 arquivos alterados e ~18 mil linhas adicionadas em relação à main. Meta: 80% de cobertura para regras de negócio críticas do backend (auth, appointment, payment, service, bot) e 60% para o frontend, alinhado ao mínimo de 80% exigido pela disciplina de Qualidade e Testes de Software; falta medir e reportar esse número formalmente (ex.: jest --coverage) na stag-facul.

9 Processamento de Linguagem Natural (PLN)

9.1. Funcionalidades de PLN: (a) Chatbot de agendamento por texto — máquina de estados (BotSessionManager) que usa um classificador de intenção para entender pedidos de agendar, consultar, cancelar ou reagendar um serviço, mantendo o contexto da conversa e acionando as mesmas regras de negócio do restante do sistema (sem IA generativa tomando ações diretamente). (b) Comando de voz — a fala é transcrita para texto (Speech-to-Text) e enviada ao mesmo chatbot, reaproveitando toda a lógica de agendamento. (c) Busca semântica de serviços — encontra serviços por significado (ex.: “preciso aparar meu cabelo” → “corte de cabelo”) mesmo sem correspondência exata de palavras.

9.2. Tecnologias e Modelos: Um microserviço Python interno (nlp-service) concentra o PLN. Pré-processamento com NLTK (normalização, remoção de acentos/URLs/menções, tokenização e stemming com SnowballStemmer em português). Classificação de intenção com TF-IDF (uni/bigramas de palavras + n-gramas de caracteres) + LinearSVC (scikit-learn); a margem de decisão da SVM define quando retornar FALLBACK. Busca semântica com embeddings do modelo multilíngue paraphrase-multilingual-MiniLM-L12-v2 (sentence-transformers) e similaridade de cosseno. Datas, horários, IDs de agendamento e nomes de serviço são extraídos por regras determinísticas no backend Node (não é NER estatístico). O comando de voz usa um provedor externo compatível com a API de transcrição da OpenAI (endpoint configurável, ex.: whisper-1).

9.3. Dataset e Treinamento: O classificador de intenções é treinado com um corpus próprio (intents.json) de frases de exemplo por intenção (agendar, cancelar, consultar, saudação etc.), curado pela equipe a partir de mensagens esperadas de clientes reais do DelBicos, complementado por regras explícitas para comandos curtos e

inequívocos (ex.: “quero agendar”, “reiniciar”). A avaliação hoje é feita observando a taxa de FALLBACK em conversas de teste manual e ajustando o limiar de confiança e o corpus; não há ainda um split formal de treino/teste publicado — recomenda-se registrar acurácia e F1-score por intenção como próximo passo de qualidade.

10 Integração entre Componentes

10.1. Fluxo de Dados: App (mobile/web) → REST/JWT + Socket.IO → API Express (Node/TS) → PostgreSQL (dados relacionais) e MongoDB (chat/notificações); a API Node consulta o nlp-service (Python) internamente via HTTP só para classificar intenção ou calcular similaridade semântica, nunca expondo o serviço Python à internet. Pagamentos passam pela API → Stripe → confirmação → atualização do Appointment. Mudanças de status de agendamento disparam eventos Socket.IO para os dispositivos conectados do cliente e do profissional, e o catálogo de serviços é atualizado no app via SSE.

10.2. Contratos de Interface: JSON sobre HTTP/HTTPS para a API REST; WebSocket (Socket.IO) para eventos em tempo real (chat, status de agendamento); SSE (text/event-stream) para atualizações do catálogo de serviços. Os contratos de payload seguem os tipos TypeScript compartilhados por convenção entre frontend e backend (src/interfaces, src/types) e são documentados via Swagger/OpenAPI (/docs) para os endpoints REST.

11 Riscos e Mitigações

11.1. Identificação de Riscos: (1) A integração do reconhecimento de voz/PLN com a API de agendamento pode gerar falsos positivos de intenção, levando a agendamentos indesejados. (2) A stag-facul (base da entrega) ainda diverge da main de produção (155 commits à frente no backend, 98 no frontend), então uma eventual promoção para produção exigirá um merge cuidadoso. (3) Middlewares de segurança (helmet, rate limiting global, sanitização) estão implementados mas desativados em produção, expondo a API a ataques comuns. (4) Falta medir formalmente a cobertura de testes da stag-facul (ver seção 8.3) para comprovar a meta de 80%/60% exigida. (5) Dependência de um provedor externo de transcrição de voz (custo e disponibilidade). (6) Prazo curto do semestre para migrar a hospedagem para Azure sem interromper o ambiente atual (AWS EC2).

11.2. Plano de Mitigação: (1) Manter sempre uma confirmação explícita do usuário antes de efetivar qualquer agendamento/cancelamento originado por voz ou chatbot (já implementado no fluxo de estados do bot). (2) Tratar a stag-facul como a branch de integração oficial até a entrega, e planejar depois um merge revisado para main antes de ir para produção real. (3) Reativar os middlewares de segurança já existentes (security.middleware.ts) e validar em homologação antes do deploy final. (4) Rodar jest --coverage na stag-facul e documentar o número real antes da entrega, endereçando lacunas encontradas. (5) Definir um provedor de transcrição de fallback e degradar graciosamente para entrada por texto quando a voz falhar. (6) Migrar para Azure em

paralelo ao ambiente atual (blue-green), trocando o domínio só após validação completa.

12 Itens do Product Backlog (PBI)

Os itens do Product Backlog (RF01–RF10 e RNFs correspondentes) estão listados nas seções 3.1 e 3.2 deste documento, na ordem de prioridade definida pela equipe (Gestão de Bicos e Agenda como prioridade mais alta, seguidos de Chat, Segurança e Relatórios). A versão original do backlog está no arquivo docs/RF_RNF.pdf; as principais diferenças em relação à versão inicial são a adição das funcionalidades de PLN/voz e do chat em tempo real, incorporadas para atender à disciplina satélite de Processamento de Linguagem Natural.

13 Anexos

13.1. Backlog do Produto: docs/RF_RNF.pdf (tabela oficial de Requisitos Funcionais e Não Funcionais) e backend/docs/RF03_RF04_PLAN.md (plano de implementação do CRUD de serviços e do filtro por raio de atuação), no repositório do projeto.

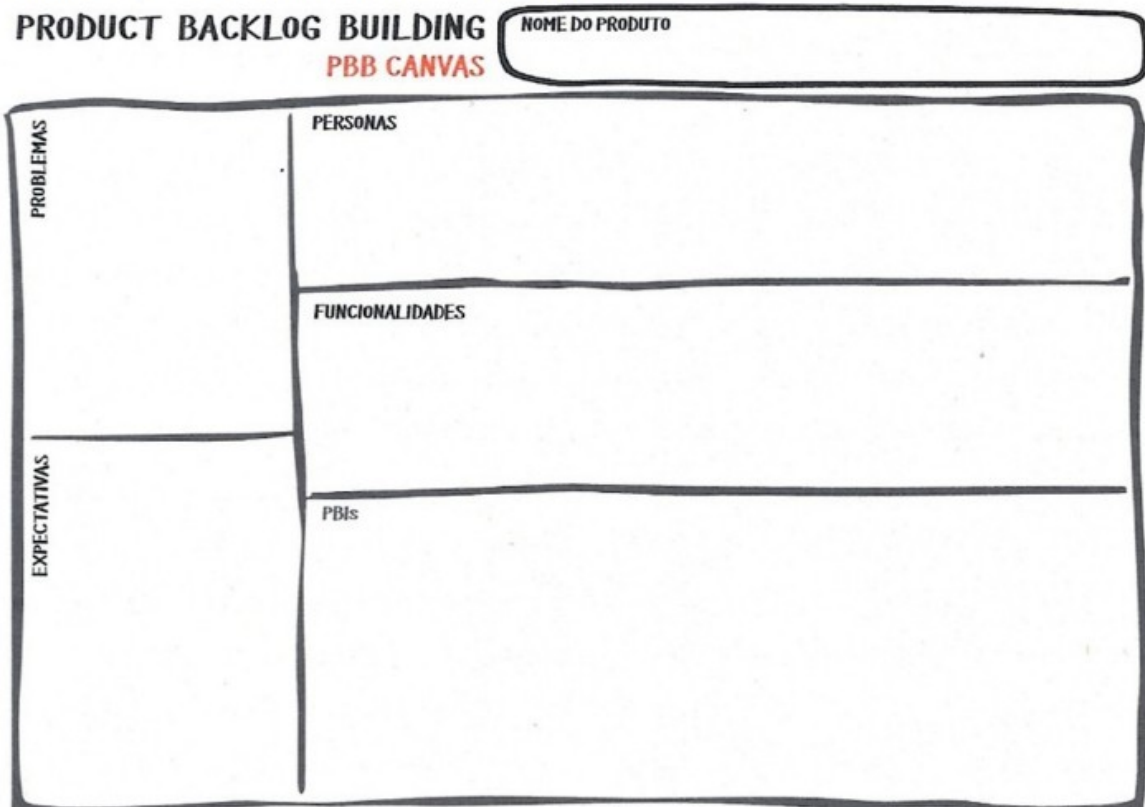
13.2. Outros Diagramas: Recomenda-se anexar aqui: diagrama de casos de uso (Cliente, Profissional, Admin), diagrama de sequência do fluxo de agendamento + pagamento, diagrama de estados do agendamento (RF05, também exigido pela disciplina de Qualidade e Testes) e o DER exportado do banco (seção 5.1).

13.3. Referências: Documentação oficial Express.js (expressjs.com), Sequelize (sequelize.org), React Native (reactnative.dev), Expo (docs.expo.dev), Stripe API (stripe.com/docs/api), scikit-learn (scikit-learn.org), Sentence-Transformers (sbert.net); documentação interna do projeto: docs/CHATBOT_PLN.md, docs/DOCUMENTACAO_TECNICA_PLN_CHATBOT.md e docs/MVP_COMANDO_VOZ_BUSCA_SEMANTICA_BACKEND.md (branch stag-facul).

14 Informações adicionais

14.1. Modelo de Product Backlog (Aguiar; Caroli, 2021, p. 51): [Imagem]

14.1. Modelo de *Product Backlog* (Aguiar; Caroli, 2021, p. 51):



14.2. Referências usadas neste modelo AGUIAR, F.; CAROLI, P. Product Backlog Building – Um guia prático para criação e refinamento de backlog para produtos de sucesso. Rio de Janeiro: ed. Caroli, 2021. DYNOWSKI, L.; DULAK, M. Dominando estilos de APIs – Compreendendo as vantagens e desvantagens dos principais estilos de APIs e escolhendo as soluções corretas. São Paulo: Novatec, 2025 "Este documento é a base para o desenvolvimento do Projeto Integrador. Mantenha-o atualizado e utilize-o como guia para todas as decisões técnicas e de planejamento."